

学号： 2104230414

沈阳建筑大学

上机报告

课程名称： 数据结构

项目名称： 链表的基本操作与栈的应用

2023年春季学期

上机时间：第 14 周 星期 四 第 5 ~ 8 节

学生姓名： 张清晨 专业班级： 计算机 2101 班

指导教师： 孙焕良 成 绩：

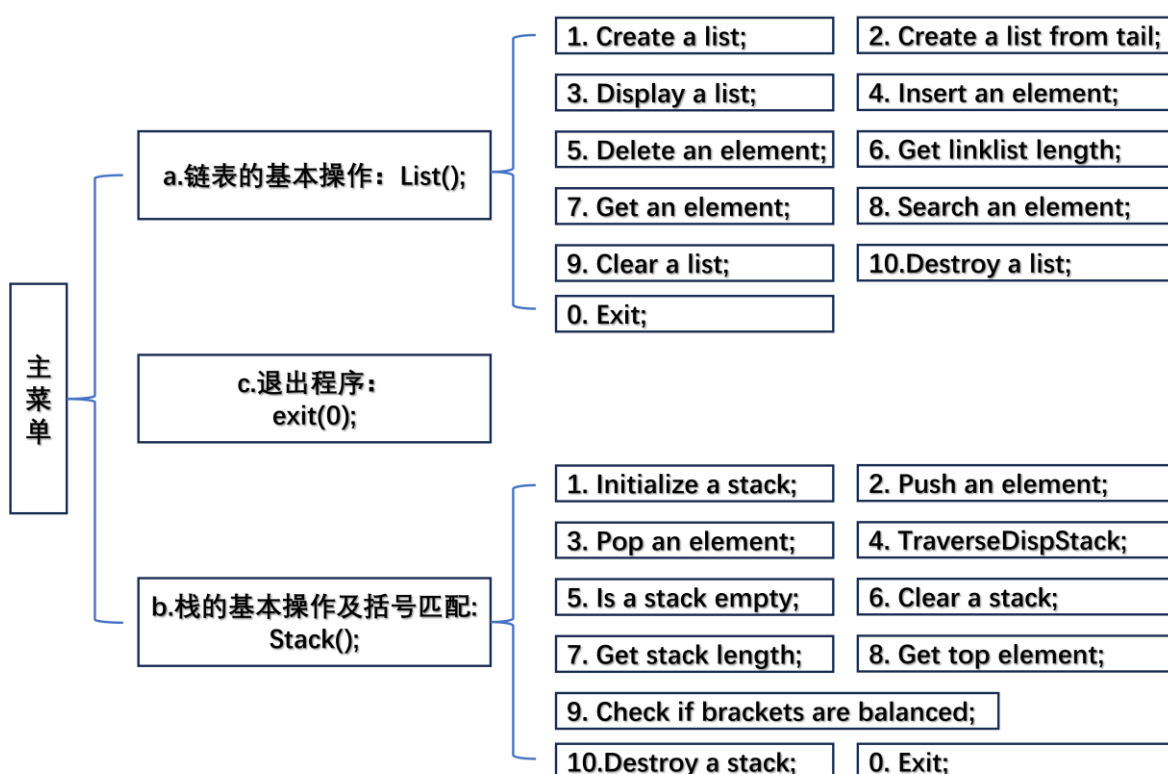
项目名称	链表的基本操作与栈的应用		学时	4
日期：2023 年 6 月 8 日		第 14 教学周		
硬件及软件				
名 称		版本、型号		
Visual C++		6.0 及以上		
上机目的与要求： 1、设计并实现链表的插入、删除、查找、求长度等基本操作； 2、利用栈的基本操作，求解迷宫、括号匹配等应用问题； 3、有菜单，显示友好； 4、报告第 1-2 页、最后一页及部分截图。 上机任务： 1、题目； 2、画出基本结构及操作原理示意图； 3、程序清单：要求对类型定义、主菜单、函数定义等内容给出注释。 4、部分结果截图。 5、结果分析和总结：最后一页				

一、 题目简介

设计并实现链表的头插法创建单链表、尾插法创建单链表、打印单链表元素、在 d 位置之前插入 e 元素、删除 d 位置的元素、输出单链表长度、取单链表 d 位置的元素、查找单链表 e 元素的位置、清空单链表、销毁单链表等基本操作；

设计并实现顺序栈的初始化一个顺序栈、从栈顶压入一个元素、从栈顶弹出一个元素、打印顺序栈的元素、判断顺序栈是否为空、清空顺序栈、获得顺序栈的长度、取栈顶元素、判断输入字符串括号是否平衡、销毁顺序栈等基本操作；并求解括号匹配的应用问题。

二、 程序实现流程图和基本结构及操作原理示意图



程序实现流程图

三、 程序清单

```
/*
数据结构上机 1   2023-6-8   计算机 2101 张清晨
*/

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>

#define TRUE 1
#define FALSE 0
#define OK 1
#define ERROR 0
#define INFEASIBLE -1
#define OVERFLOW -2

#define STACK_INIT_SIZE 100 //栈初始开辟长度
#define STACKINCREMENT 10 //栈空间满时的增量

typedef int Status; //定义别名
typedef int ElemType;
typedef char SElemType;

typedef struct LNode{ //链表的表示
    ElemType data; //单链表的数据域
    struct LNode *next; //单链表的指针域
}LNode, *LinkList;

typedef struct{ //顺序栈的表示
    SElemType *base; //顺序栈的栈底指针
    SElemType *top; //顺序栈的栈顶指针
    int stacksize; //顺序栈的长度
}SqStack,*PSqStack;

/*单链表函数声明*/
void List(); //链表的基本操作菜单
LinkList CreatList_L(LinkList L,int n); //头插法创建链表
LinkList CreatList_LT(LinkList L, int n); //尾插法创建链表
Status DispList_L(LinkList L); //打印链表元素
Status ListInsert_L(LinkList L,int i, ElemType e); //在第 i 位置之前插入一个元素
Status ListDelete_L(LinkList L,int i); //删除第 i 个位置的元素，并返回删除的值
LNode * GetElem_L(LinkList L,int i); //取第 i 个位置元素的值
Status ClearList_L(LinkList L); //清空链表
Status DestroyList_L(LinkList L); //销毁链表
int ListLength_L(LinkList L); //获得链表的长度
int Search_L(LinkList L,ElemType e); //插好第一个与 e 相同元素的位置
```

```

/*顺序栈函数声明*/
void Stack(); //栈的基本操作
Status InitStack(PSqStack S); //初始化栈
Status ClearStack(PSqStack S); //清空栈
Status DestroyStack(PSqStack S); //销毁栈
Status StackEmpty(PSqStack S); //判断栈是否为空
int StackLength(PSqStack S); //取栈的长度
Status Push(PSqStack S, SElemType e); //在栈顶插入元素
SElemType Pop(PSqStack S); //在栈顶删除元素
SElemType GetTop(PSqStack S); //取栈顶元素
Status StackTraverse(PSqStack S); //输出栈中元素
int BracketCheck(char *a); //字符串括号匹配函数

int main()
{
    char c;

    while(1){ //主菜单
        printf("请选择以下三种操作之一： \n");
        printf("a.链表的基本操作\n");
        printf("b.栈的基本操作及括号匹配\n");
        printf("c.离开\n");

        scanf(" %c",&c);

        switch(c)
        {
            case 'a':
                List();
                break;
            case 'b':
                Stack();
                break;
            case 'c':
                exit(0);
            default:
                printf("Print an incorrect letter;\n");
                break;
        }

        return 0;
        system("PAUSE");
    }
}

```

```

/*单链表的基本操作*/
void List()
{
    int c;
    int e,d,n;
    LinkList L1;
    L1 = (LinkList)malloc(sizeof(LNode));

    while(1){
        printf("\nList Implementation by Sequence\n");
        printf("=====\\n");
        printf("1. Create a list;      2. Create a list from tail; \\n");
            //1.头插法创建单链表      2.尾插法创建单链表
        printf("3. Display a list;      4. Insert an element; \\n");
            //3.打印链表      4.向链表中插入元素
        printf("5. Delete an element; 6. Get linklist length; \\n");
            // 5.删除链表中的一个元素      6.获得链表的长度
        printf("7. Get an element;      8. Search an element;\\n");
            //7.取某一个位置上的元素      8.查找元素位置
        printf("9. Clear a list;      10.Destroy a list;\\n");
            //9.清空链表      10.销毁链表
        printf("0. Exit; \\n");
        printf("=====\\n");

        scanf(" %d",&c);//fflush(stdin);

        switch(c){

            case 1: //头插法创建单链表
                printf("Input the number of nodes:\\n");
                scanf(" %d",&n);
                L1 = CreatList_L(L1,n);
                break;
            case 2: //尾插法创建单链表
                printf("Input the number of nodes:\\n");
                scanf(" %d", &n);
                L1 = CreatList_LT(L1, n);
                break;
            case 3: //打印单链表元素
                DispList_L(L1);
                break;
            case 4: //在 d 位置之前插入 e 元素
                printf("Input a location and an element:\\n");
                scanf("%d%d",&d,&e);
                if(ListInsert_L(L1,d,e))
                    printf("insert successfully!\\n");
                else
                    printf("insert failure!\\n");
                break;

```

```

case 5: //删除 d 位置的元素
    printf("Input a location:\n");
    scanf(" %d",&d);
    ListDelete_L(L1,d);
    break;
case 6: //输出单链表长度
    printf("list length is %d\n",ListLength_L(L1));
    break;
case 7: //取单链表 d 位置的元素
    printf("Input a location to get an element:\n");
    scanf(" %d",&d);
    if(!GetElem_L(L1,d))
        printf("Location Error!\n");
    else
        printf("The element is %d\n",GetElem_L(L1,d)->data);
    break;
case 8: //查找单链表 e 元素的位置
    printf("Input an element to get a location:\n");
    scanf(" %d",&e);
    if(Search_L(L1,e))
        printf("The location is %d\n",Search_L(L1,e));
    else
        printf("Do not find this element!\n");
    break;
case 9: //清空单链表
    if (ClearList_L(L1))
        printf("clear successfully!\n");
    else
        printf("clear failure!\n");
    break;
case 10: //销毁单链表
    if (DestroyList_L(L1))
        printf("destroy successfully!\n");
    else
        printf("destroy failure!\n");
    break;
case 0: //结束程序
    exit(0);
default: //错误输入
    printf("Print an incorrect letter;\n");
    break;
}

}

system("PAUSE"); //防止程序一直运行
}

```

```
/*用头插法创建一个单链表*/
```

```
LinkedList CreatList_L(LinkedList L,int n)
```

```
{
    int i;
    LNode *p;

    L = (LinkedList )malloc(sizeof(LNode));
    L->next = NULL;

    for(i=n; i > 0 ; --i) {
        p = (LinkedList )malloc(sizeof(LNode));
        printf("Please input the %d-th element:\n",n-i+1);
        scanf("%d",&p->data);
        p->next = L->next;
        L->next = p;
    }
    printf("A linked list with %d elements has been created!\n", n);
    return L;
}
```

```
/*用尾插法创建一个单链表*/
```

```
LinkedList CreatList_LT(LinkedList L, int n)
```

```
{
    int i;
    LNode* p, *r;
    L = (LinkedList)malloc(sizeof(LNode));
    r = L;
    L->next = NULL;

    for (i = n; i > 0; --i) {
        p = (LinkedList)malloc(sizeof(LNode));
        printf("Please input the %d-th element:\n", n - i + 1);
        scanf("%d", &p->data);
        r->next = p;
        r = p;
    }
    r->next = NULL;
    printf("A linked list with %d elements has been created!\n", n);
    return L;
}
```

```
/*打印一个单链表各元素*/
```

```
Status DispList_L(LinkedList L)
```

```
{
    LNode *p=L;
    if (p==NULL)
    {   printf("A list has not be created!\n");
        return 0;
    }
}
```



```

    p=p->next;
    if (p == NULL)
    {
        printf("A null list!\n");
        return 0;
    }
    while (p!=NULL)
    {
        printf("%5d", p->data);
        p=p->next;
    }
    printf("\n");
    return OK;
}

/*在第 i 个位置前插入元素 e*/
Status ListInsert_L(LinkList L,int i, ElemType e)
{
    int j = 0;
    LNode *p, *s;
    p = L;
    while (p->next != NULL && j < i - 1)
    {
        p = p->next;
        j++;
    }
    if (p != NULL)
    {
        s = (LinkList) malloc(sizeof(LNode));
        s->data = e;
        s->next = p->next;
        p->next = s;
        return OK;
    }

    return ERROR;
}

/*删除第 i 个位置元素*/
Status ListDelete_L(LinkList L,int i)
{
    int j = 0,e;
    LNode *p, *s;
    p = L;
    while (p->next != NULL && j < i - 1) {
        p = p->next;
        j++;
    }

```

```

        if (p->next != NULL && j == i - 1) {
            s = p->next;
            e=s->data;
            p->next = s->next;
            free(s);
            printf("delete element %d successfully!\n",e);
        }
        else
        {
            printf("delete failure!\n");
        }
        return OK;
    }
}

```

/*返回第 i 个位置元素*/

LNode *GetElem_L(LinkList L, int i)

```

{
    LNode *p;
    p = L->next;
    int j = 1;
    if (p == NULL){
        printf("A null list\n");
        return 0;
    }
    while (p && j < i){
        p = p->next;
        j++;
    }
    if (!p || j > i) return ERROR;
    else return p;
}

```

/*清空一个单链表*/

Status ClearList_L(LinkList L)

```

{
    LNode *s;
    if (L == NULL)
    {
        printf("A null list!\n");
        return 0;
    }
    while (L->next !=NULL)
    {
        s = L->next;
        L->next = s->next;
        free(s);
    }
    return OK;
}

```

/*销毁一个单链表*/

Status DestroyList_L(LinkList L)

```
{
    LNode *p;
    if (L == NULL)
    {
        printf("A null list!\n");
        return 0;
    }
    while(L){
        p= L;
        L=L->next;
        free(p);
    }
    return OK;
}
```

/*获得一个单链表的长度*/

int ListLength_L(LinkList L)

```
{
    LNode *p;
    int i=0;
    p=L->next;
    while(p){
        i++;
        p=p->next;
    }

    return i;
}
```

/*按值查找：查找一个单链表的元素，并返回其位置*/

int Search_L(LinkList L,ElemType e)

```
{
    LNode *p;
    int i=1;
    p=L->next;
    while(p && p->data!=e)
    {
        p=p->next;
        i++;
    }
    if(p) return i;
    else return 0;
}
```

```

/*顺序栈的基本操作和括号匹配函数*/
void Stack()
{
    int c;
    char e;
    char str[100];
    SqStack S1;

    while(1){
        printf("\nStack Implementation by Sequence\n");
        printf("=====\\n");
        printf("1. Initialize a stack; 2. Push an element;\\n");
            //1.初始化顺序栈    2.压入一个元素
        printf("3. Pop an element;    4. TraverseDispStack;\\n");
            //3.弹出一个元素    4.打印栈中的元素
        printf("5. Is a stack empty;    6. Clear a stack;\\n");
            // 5.判断栈是否为空    6.清空栈
        printf("7. Get stack length;    8. Get top element;\\n");
            //7.获得栈的长度    8.取栈顶元素
        printf("9. Check if brackets are balanced;    \\n");
            //9.判断字符串的括号是否平衡
        printf("10.Destroy a stack;    0. Exit;\\n"); //10.销毁栈
        printf("=====\\n");

        scanf(" %d",&c);

        switch(c){
            case 1: //初始化一个顺序栈
                InitStack(&S1);
                printf("initialize successfully!\\n");
                break;
            case 2: //从栈顶压入一个元素
                printf("Input an element:\\n");
                scanf(" %c",&e);
                if(Push(&S1, e))
                    printf("push successfully!\\n");
                else
                    printf("push failure!\\n");
                break;
            case 3: //从栈顶弹出一个元素
                if(Pop(&S1))
                    printf("Pop the element %c\\n",Pop(&S1));
                else
                    printf("This is an empty stack!\\n");
                break;
            case 4: //打印顺序栈的元素
                StackTraverse(&S1);
                break;

```

```

    case 5: //判断顺序栈是否为空
        if(StackEmpty(&S1))
            printf("Empty!\n");
        else
            printf("Not Empty!\n");
        break;
    case 6: //清空顺序栈
        ClearStack(&S1);
        printf("clear successfully!\n");
        break;
    case 7: //获得顺序栈的长度
        printf("stacklength is %d\n", StackLength(&S1));
        break;
    case 8: //取栈顶元素
        if(GetTop(&S1))
            printf("top element is %c\n", GetTop(&S1));
        else
            printf("This is an empty stack!\n");
        break;
    case 9: //判断输入字符串括号是否平衡
        printf("Input an expression\n");
        scanf(" %s", str); // gets(str);
        if (BracketCheck(str))
            printf("Balanced!\n");
        else
            printf("Not balanced!\n");
        break;
    case 10: //销毁顺序栈
        DestroyStack(&S1);
        printf("destroy successfully!\n");
        break;
    case 0: //离开程序
        exit(0);
    default:
        printf("Print an incorrect letter;\n");
        break;
}
}
system("PAUSE");
}

/*栈的初始化*/
Status InitStack(PSqStack S)
{
    S->base = (SElemType *)malloc(STACK_INIT_SIZE*sizeof(SElemType));
    if (!S->base) exit(OVERFLOW);
    S->top = S->base ;
    S->stacksize = STACK_INIT_SIZE;
    return OK;
}

```

```

/*入栈操作：从栈顶压入一个元素*/
Status Push(PSqStack S, SElemType e)
{
    if (S->top - S->base >= S->stacksize){
        S->base = (SElemType *)realloc(S->base,(S->stacksize+STACKINCREMENT)*
sizeof(SElemType));
        if (!S->base) exit(OVERFLOW);
        S->top = S->base + S->stacksize;
        S->stacksize += STACKINCREMENT;
    }
    *S->top++ = e; //等效于 *S->top=e; S->top++;

    return OK;
}

/*出栈操作：从栈顶弹出一个元素*/
SElemType Pop(PSqStack S)
{
    if (S->top == S->base) //等效于 if(StackEmpty)
        return ERROR;

    return *--S->top; //等效于 --S->top; e=*S->top; return e;
}

/*取栈顶元素的值*/
SElemType GetTop(PSqStack S)
{
    if (S->top == S->base) return ERROR;

    return *(S->top - 1);
}

/*判断栈是否空*/
Status StackEmpty(PSqStack S)
{
    if(S->top==S->base)
        return TRUE;
    else
        return FALSE;
}

```

```

/*输出栈中元素*/
Status StackTraverse(PSqStack S)
{
    SElemType *p;
    p=S->top;
    if (StackEmpty(S)){
        printf("This is an empty stack!\n");
        return ERROR;
    }
    else {
        while(p > S->base){
            printf("%5c", *--p);

        }
    }
    printf("\n");

    return OK;
}

```

```

/*返回一个栈的长度*/
int StackLength(PSqStack S)
{
    return S->top - S->base;
}

```

```

/*清空一个栈*/
Status ClearStack(PSqStack S)
{
    if(S->base)
        S->top=S->base;

    return OK;
}

```

```

/*破坏一个栈*/
Status DestroyStack(PSqStack S)
{
    if(S->base){
        free(S->base);
        S->stacksize=0;
        S->top=S->base=NULL;
    }

    return 0;
}

```

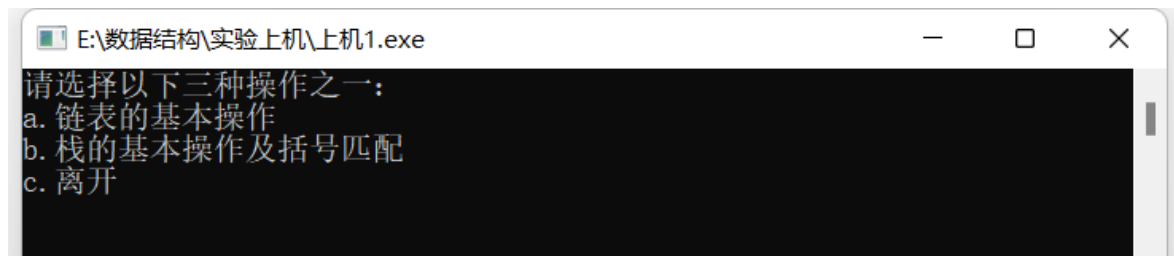
```

/*括号匹配函数*/
int BracketCheck(char *a)
{
    int i=0;
    SqStack s;
    SElemType x;
    InitStack(&s);
    while(a[i])
    {
        switch(a[i])
        {
            case '(':
                Push(&s,a[i]); //括号入栈
                break;
            case '[':
                Push(&s,a[i]); //括号入栈
                break;
            case '{':
                Push(&s,a[i]); //括号入栈
                break;
            case ')': //与栈中栈顶括号比较是否匹配，匹配弹出栈顶元素，不匹配失败
                x=GetTop(&s);
                if(x=='(')
                    Pop(&s);
                else
                    return FALSE;
                break;
            case ']': //与栈中栈顶括号比较是否匹配，匹配弹出栈顶元素，不匹配失败
                x=GetTop(&s);
                if(x=='[')
                    Pop(&s);
                else
                    return FALSE;
                break;
            case '}': //与栈中栈顶括号比较是否匹配，匹配弹出栈顶元素，不匹配失败
                x=GetTop(&s);
                if(x=='{')
                    Pop(&s);
                else
                    return FALSE;
                break;
            default:
                break;
        }
        i++;
    }
    if(s.top==s.base) //最后栈中元素为空，则匹配成功
        return TRUE;
    return FALSE;
}

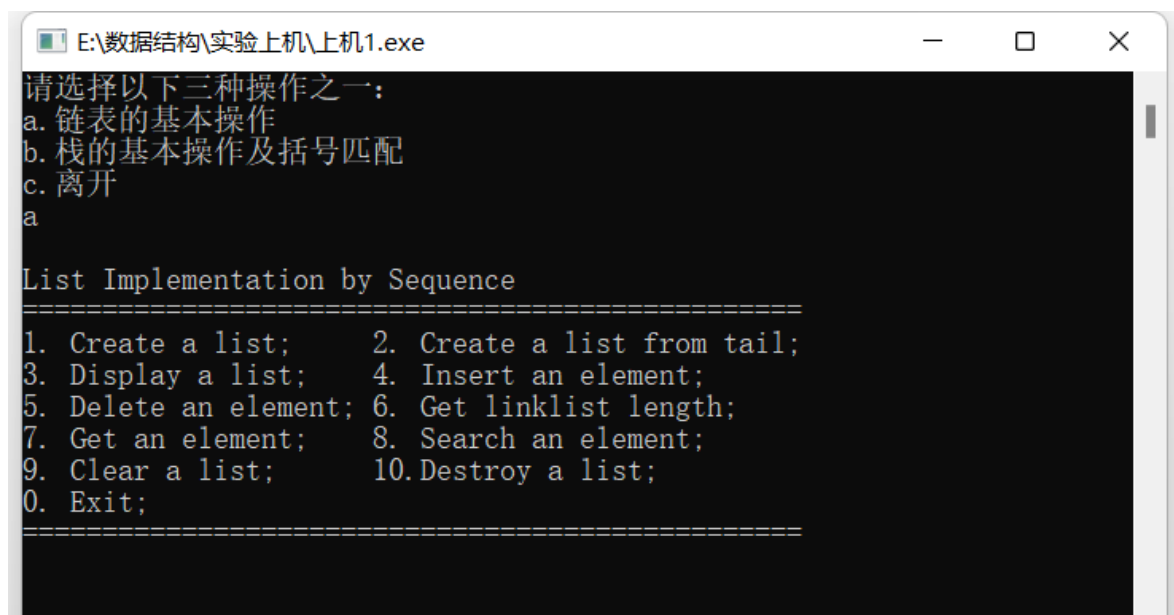
```


四、 运行截图

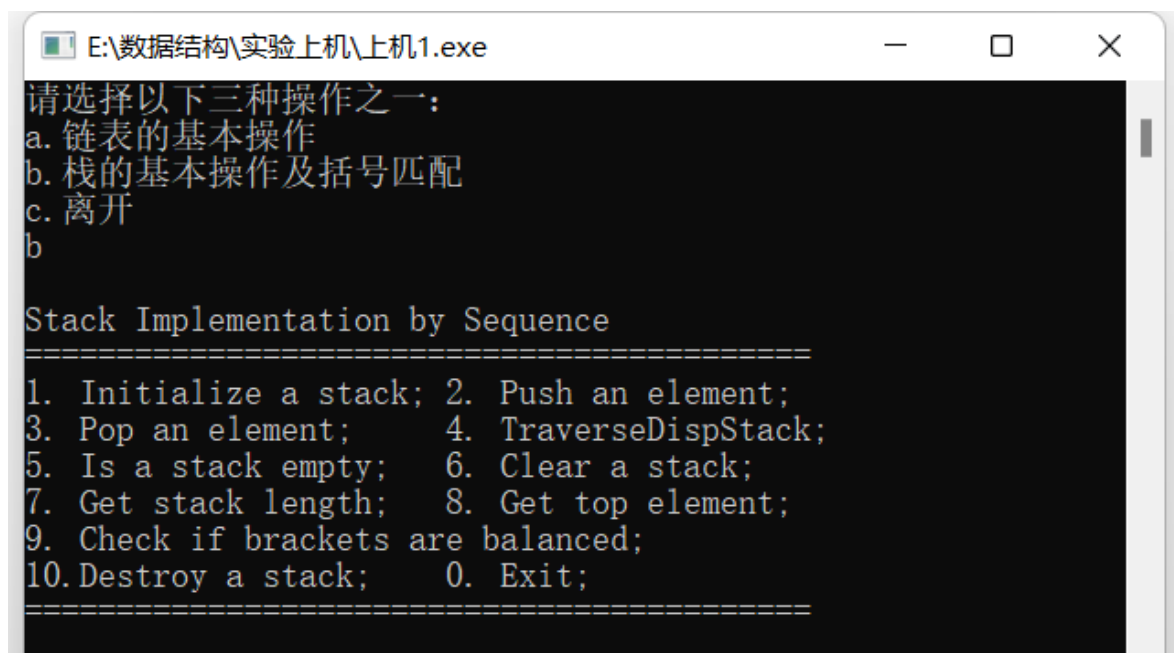
1.主菜单



2.链表基本操作菜单



3.栈的基本操作菜单



4.头插法创建链表

```
E:\数据结构\实验上机\上机1.exe
List Implementation by Sequence
=====
1. Create a list;      2. Create a list from tail;
3. Display a list;    4. Insert an element;
5. Delete an element; 6. Get linklist length;
7. Get an element;    8. Search an element;
9. Clear a list;      10. Destroy a list;
0. Exit;
=====
1
Input the number of nodes:
5
Please input the 1-th element:
6
Please input the 2-th element:
5
Please input the 3-th element:
7
Please input the 4-th element:
8
Please input the 5-th element:
9
A linked list with 5 elements has been created!
```

5.打印链表元素

```
E:\数据结构\实验上机\上机1.exe
List Implementation by Sequence
=====
1. Create a list;      2. Create a list from tail;
3. Display a list;    4. Insert an element;
5. Delete an element; 6. Get linklist length;
7. Get an element;    8. Search an element;
9. Clear a list;      10. Destroy a list;
0. Exit;
=====
3
    9    8    7    5    6
```

6. 尾插法创建链表

```
E:\数据结构\实验上机\上机1.exe
List Implementation by Sequence
=====
1. Create a list;      2. Create a list from tail;
3. Display a list;    4. Insert an element;
5. Delete an element; 6. Get linklist length;
7. Get an element;    8. Search an element;
9. Clear a list;      10. Destroy a list;
0. Exit;
=====
2
Input the number of nodes:
5
Please input the 1-th element:
1
Please input the 2-th element:
2
Please input the 3-th element:
3
Please input the 4-th element:
4
Please input the 5-th element:
5
A linked list with 5 elements has been created!

List Implementation by Sequence
=====
1. Create a list;      2. Create a list from tail;
3. Display a list;    4. Insert an element;
5. Delete an element; 6. Get linklist length;
7. Get an element;    8. Search an element;
9. Clear a list;      10. Destroy a list;
0. Exit;
=====
3
    1    2    3    4    5
```

7. 插入元素

```
E:\数据结构\实验上机\上机1.exe
4
Input a location and an element:
1
0
insert successfully!
```

8. 删除元素

```
E:\数据结构\实验上机\上机1.exe
5
Input a location:
1
delete element 0 successfully!
```

9.获得链表长度

```
E:\数据结构\实验上机\上机1.exe
6
list length is 5
```

10.获得某位置的元素

```
E:\数据结构\实验上机\上机1.exe
7
Input a location to get an element:
1
The element is 1
```

11.查到某元素的位置

```
E:\数据结构\实验上机\上机1.exe
8
Input an element to get a location:
5
The location is 5
```

12.栈的初始化

```
E:\数据结构\实验上机\上机1.exe
Stack Implementation by Sequence
=====
1. Initialize a stack; 2. Push an element;
3. Pop an element;    4. TraverseDispStack;
5. Is a stack empty;  6. Clear a stack;
7. Get stack length;  8. Get top element;
9. Check if brackets are balanced;
10. Destroy a stack;  0. Exit;
=====
1
initialize successfully!
```

13.从栈顶插入元素

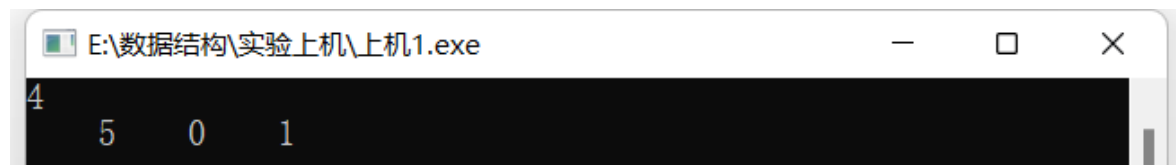
```
E:\数据结构\实验上机\上机1.exe
2
Input an element:
1
push successfully!
```

14.删除栈顶元素



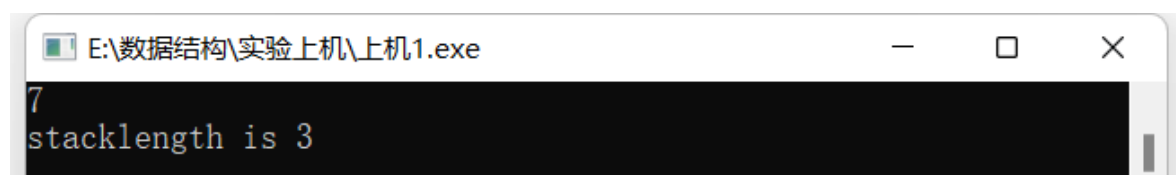
```
E:\数据结构\实验上机\上机1.exe
3
Pop the element 1
```

15.打印栈中元素



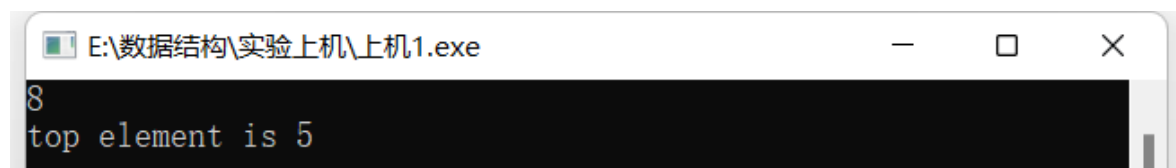
```
E:\数据结构\实验上机\上机1.exe
4
5 0 1
```

16.获得栈的长度



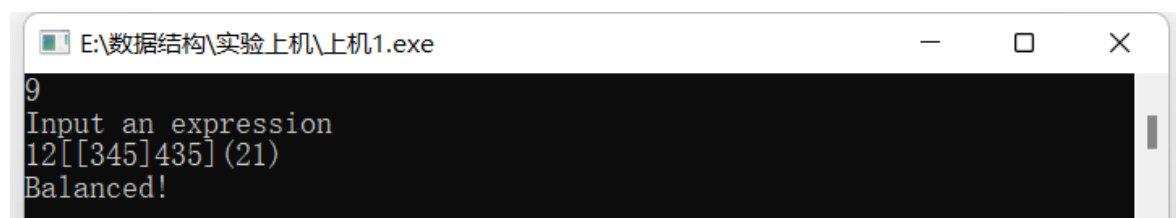
```
E:\数据结构\实验上机\上机1.exe
7
stacklength is 3
```

17.获得栈顶的元素



```
E:\数据结构\实验上机\上机1.exe
8
top element is 5
```

18.括号匹配



```
E:\数据结构\实验上机\上机1.exe
9
Input an expression
12[[345]435] (21)
Balanced!
```

五、 结果分析和总结：

运行程序后，在主菜单内选择进行链表的操作还是栈的操作，选择链表的基本操作后，会看到链表的菜单，可以选择通过头插法或者尾插法来创建一个单链表，然后可以打印链表内元素，也可以在某一位置之前插入一个元素或删除某一位置的元素，通过单链表的指针往后移动也可以得到单链表的长度，通过单链表的数据域可以取某一位置的元素，也可以查找某一元素的位置，最后也能对链表进行清除和销毁操作。选择栈的基本操作及括号匹配后，可以进入栈的菜单，先对栈进行初始化，然后可以在栈顶进行 push 插入元素或 pop 删除元素，也可以将栈中的元素依次打印，同时还可以判断栈是否为空，以及获得栈的长度、栈定元素，其次也有清除或销毁一个栈。字符串括号匹配，通过输入一个字符串来判断其括号是否平衡。

通过第一次上机实验报告的书写，加深了我对链表和栈的理解与记忆，同时也掌握了栈的应用之一：字符串括号匹配问题；在代码的书写过程中我也遇到了很多问题、警告和报错，通过查阅相关资料及询问同学，最终都得以解决。同时我认为很重要的一点是要学会自己书写代码，而不能仅仅是能看懂然后对着书本敲！这一点我还需要加强。

本 次 上 机 成 绩	项目	操作	报告书写	出勤和课堂纪律	其他
	得分				
	成绩合计：				

评语：

教师签字：

日期：